

Comparison between Baseline (Logistic Regression), Perceptron and MLP from scratch.

This project compares the performance of different models in classifying diabetes based on biological data. The models that are being trained and compared are:

- Logistic Regression
- Perceptron
- MLP from scratch

The objective is to understand how complexity of neural networks and hyperparameter tuning contribute to classification accuracy and learning capability.

Dataset

Dataset titled Diabetes Dataset from kaggle, [Diabetes Dataset | Kaggle](#), is used in this assignment. This dataset contains 768 instances with 8 attributes and one target. The attributes will be used to make predictions on classifying the instances into 1 or 0, indicating the presence or absence of Diabetes.

Data Preprocessing

```
Pregnancies      0
Glucose          0
BloodPressure    0
SkinThickness    0
Insulin          0
BMI              0
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64
```

The missing value for each attribute is determined using python, and the number of missing values are printed out. Based on the result, the dataset does not contain any missing value. Therefore, the dataset can proceed to the next process.

Standardization is a method to rescale the value of attributes to a smaller range, to ease the process of modelling further. The mean is rescaled to 0 and standard deviation is rescaled to 1. The data produced is used for modelling purposes.

Next, the data is splitted into two sets, training sets and testing sets. 80% of the data is splitted for training and 20% for testing purposes.

Baseline

The pre-processed data was then used to train logistic regression, which serves as the baseline model.

Logistic regression			
	precision	recall	f1-score
0	0.81	0.80	0.81
1	0.65	0.67	0.66
Accuracy			0.75
Macro avg	0.73	0.74	0.73
Weighted avg	0.76	0.75	0.75

Classification report for Logistic Regression

Perceptron

The Perceptron model is then used to classify diabetes. The modelling of perceptron is using default value, in which by default, the learning rate is set to 1.0, max_iter is set to 100, alpha is set to 0.0001 and penalty set to l2. The accuracy score is evaluated to view the performance of the model.

```
#modelling
perceptron = Perceptron(random_state=42, penalty= 'l2' , alpha=0.0001, max_iter=1000,
eta0=1.0)

perceptron.fit(X_train,y_train)

y_pred = perceptron.predict(X_test)
```

The hyperparameters of the perceptron are tuned to improve the performance of the model, to be specific, to increase the accuracy of the prediction. The hyperparameters chosen are learning rate, regularization, maximum iterations and penalty.

Learning rate is the speed of learning, affecting the speed of the learning process of a model, the higher the value of learning rate, the faster the model learns.

Regularization is the fitting of the training data, in this case, the lower the value, the more the model prioritises the fitting of data. Maximum iterations is the maximum number of iterations of the process to allow the repetitive training process to be conducted without stopping prematurely.

Penalty, meanwhile, is the type of regularization that is in charge of the overfitting of the model.

The values of each hyperparameter are tuned to produce a better model. The results are summarized as below.

	penalty	alpha	max_iter	eta0	Accuracy
Model 1	l2	0.0001	1000	1.0	0.71
Model 2	l2	0.0001	1000	5.0	0.71
Model 3	l2	0.0001	500	1.0	0.71
Model 4	l2	0.0001	500	2.0	0.71
Model 5	l2	0.00001	1000	1.0	0.73
Model 6	l2	0.00001	1000	5.0	0.72
Model 7	l2	0.001	1000	1.0	0.69
Model 8	l2	0.001	1000	2.0	0.68
Model 9	l2	0.001	500	2.0	0.68
Model 10	l1	0.0001	1000	1.0	0.51
Model 11	l1	0.0001	1000	5.0	0.51
Model 12	l1	0.0001	500	1.0	0.51
Model 13	l1	0.00001	1000	1.0	0.58
Model 14	l1	0.00001	1000	2.0	0.58
Model 15	l1	0.001	1000	1.0	0.73
Model 16	l1	0.001	1000	2.0	0.73
Model 17	l1	0.001	500	1.0	0.73
Model 18	l1	0.001	200	5.0	0.73

Model Evaluation

Classification Report

The models with similar results are further analysed in this section by comparing the precision, recall and f1 score.

Model	penalty	alpha	max_iter	eta0	Accuracy
Model 5	l2	0.00001	1000	1.0	0.73
Model 6	l2	0.00001	1000	5.0	0.72
Model 15	l1	0.001	1000	1.0	0.73
Model 16	l1	0.001	1000	2.0	0.73
Model 17	l1	0.001	500	1.0	0.73
Model 18	l1	0.001	200	5.0	0.73

The classification report for each model is tabulated.

Model 5			
	precision	recall	f1-score
0	0.78	0.81	0.80
1	0.63	0.60	0.62
Macro avg	0.71	0.70	0.71
Weighted avg	0.73	0.73	0.73
Confusion matrix	[[80 19] [22 33]]		

Model 6			
	precision	recall	f1-score
0	0.82	0.73	0.77
1	0.59	0.71	0.64
Macro avg	0.70	0.72	0.71
Weighted avg	0.74	0.72	0.73
Confusion matrix	[[72 27] [16 39]]		

Model 15			
	precision	recall	f1-score
0	0.81	0.75	0.78
1	0.60	0.69	0.64
Macro avg	0.71	0.72	0.71
Weighted avg	0.74	0.73	0.73
Confusion matrix	[[74 25] [17 38]]		

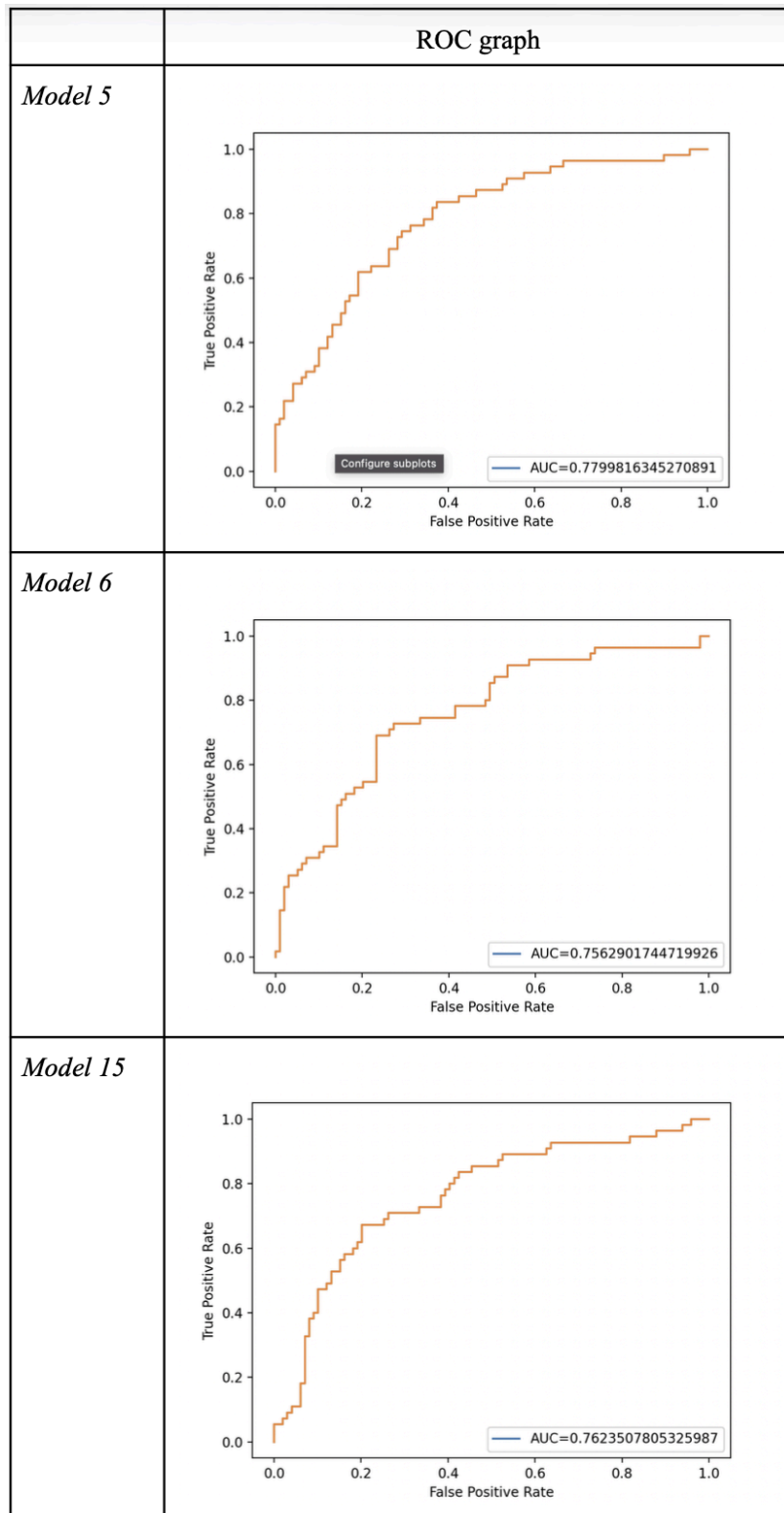
Model 16			
	precision	recall	f1-score
0	0.81	0.75	0.78
1	0.60	0.69	0.64
Macro avg	0.71	0.72	0.71
Weighted avg	0.74	0.73	0.73
Confusion matrix	[[74 25] [17 38]]		

Model 17			
	precision	recall	f1-score
0	0.81	0.75	0.78
1	0.60	0.69	0.64
Macro avg	0.71	0.72	0.71
Weighted avg	0.74	0.73	0.73
Confusion matrix	[[74 25] [17 38]]		

Model 18			
	precision	recall	f1-score
0	0.81	0.75	0.78
1	0.60	0.69	0.64
Macro avg	0.71	0.72	0.71
Weighted avg	0.74	0.73	0.73
Confusion matrix	[[74 25] [17 38]]		

ROC

Model 5, 6 and 15 are used to test their ROC only, as the performance of model 16, 17 and 18 is the same as model 15 based on classification reports.



AUC rate of 0.76 shows that both the model 6 and 15 are performing well in classification of this dataset, proven by the ROC curve that is close to the left top. Model 5 has a slightly higher AUC rate of 0.78, reflecting the slightly better performance of this model compared to the others.

Moving to the classification report, the value of max_iter does not change the accuracy and f1-score much, suggesting that the model is working well across different numbers of iterations. However, the increase of learning rate from 1.0 in model 5 to 5.0 in model 6 lead to a slight decrease of accuracy, proving that faster learning process may bring variability to the performance.

Based on the classification reports on different models, the best model is model 6, with highest precision for class 0 among all models, indicating lesser false positive predicted compared to other models. On the other hand, it has the least false negative among the models, reaching the highest recall value for class 1 with value 0.71.

Furthermore, both precision and recall values show that model 6 produces the least false positive and false negative compared to other models, revealing the ability to balance in minimising both false positive and false negative. This is significant to minimise the fault from the prediction as well as providing a reliable result. Model 15, 16, 17 and 18 produce a similar classification report, nevertheless, among all models, the macro avg and weighted avg are identical, suggesting that the performance is consistent for these models. Model 5 has the lowest precision for class 0 and recall for class 1, although having higher accuracy, higher predictions of false negative and false positive may lead to incorrect results.

In conclusion, model 6 is chosen due to its comprehensive analysis of the classification report and ROC, despite not having the highest accuracy and AUC, this model is able to provide reliable results.

Comparison with Baseline

The performance of Logistic Regression and Perceptron was compared based on the classification report.

Logistic regression			
	precision	recall	f1-score
0	0.81	0.80	0.81
1	0.65	0.67	0.66
Accuracy			0.75
Macro avg	0.73	0.74	0.73
Weighted avg	0.76	0.75	0.75

Model 6 (Perceptron)			
	precision	recall	f1-score
0	0.82	0.73	0.77
1	0.59	0.71	0.64
Accuracy			0.75
Macro avg	0.70	0.72	0.71
Weighted avg	0.74	0.72	0.73

Compared with model 6, the precision for class 0 using logistic regression is slightly lower. The recall value for class 1 in model 6 is higher than using logistic regression, indicating that model 6 predicts less false negatives, meaning that it is able to predict more positive cases. In the medical field, the minimizing of false negatives is crucial to avoid any missing positive cases that could bring critical impacts. Ergo for this dataset, determining the presence of diabetes is the purpose, model 6 would be better to be used for prediction, even though the accuracy is not as high as logistic regression.

Perceptron is the simple model of neural networks, it can handle large amounts of data and provide a faster way to predict the binary outcome, due to the simplicity of the model, it is efficient in classifying samples into two categories. It is easy to train and perform well in separating linear vectors.

However, one of the limitations of perceptron is that the output is limited to binary numbers. Non-linear vectors can cause poor performance. Using a perceptron is a simple model, therefore cannot deal with complex tasks, such as identifying the relationships between two attributes. As the number of inputs increases, the model may not be able to produce an accurate result due to the simplicity of the model.

This dataset consists of 8 attributes that contribute to the outcome, it is possible to have relationships between each attribute. The complex pattern between attributes is unable to detect by using a perceptron, hence, logistic regression seems to perform well here. Consider using a multilayer perceptron to make the model more suitable to produce a better result, minimizing the false positive and false negative results at the same time.

Multi-layer perceptron (MLP)

Multi-layer perceptron (MLP) was then used to classify the data, which consists of an input layer, hidden layer and output layer.

The performance of different combinations of hyperparameters values will be evaluated to determine the best hyperparameters.

Model Implementation

The MLP model is implemented using python from scratch. The architecture of MLP includes input, hidden neurons and output. Here, the MLP implemented is only with one hidden layer. The number of hidden neurons was initially set to 2 and tuning of this model will be discussed.

The number of input neurons is the number of attributes from the dataset, which is 8, with one output neuron as this is a binary classification that produces only one output, either 1 or 0.

Forward Pass and Backpropagation

Forward pass and backpropagation is implemented and aims to update the weight. The activation was set initially to sigmoid and will be tuned.

```
def feedforward(self, x):
    z1 = self.activation(np.dot(x, self.w1) + self.b1)
    y = sigmoid_act(np.dot(z1, self.wOut) + self.bOut)
    return z1, y
def backpropagation(self, x, z1, y, target):
    loss = -target * np.log(y) - (1 - target) * np.log(1 - y)
    loss = np.mean(loss)
    #delta
    delta_out = (y - target)
    delta_hidden = np.dot(delta_out, self.wOut.T) * self.activation(z1, der=True)
    # Update weights and biases
    self.wOut -= self.eta * np.dot(z1.T, delta_out)
    self.bOut -= self.eta * np.sum(delta_out, axis=0, keepdims=True)
    self.w1 -= self.eta * np.dot(x.reshape(1,-1).T, delta_hidden)
    self.b1 -= self.eta * np.sum(delta_hidden, axis=0, keepdims=True)
```

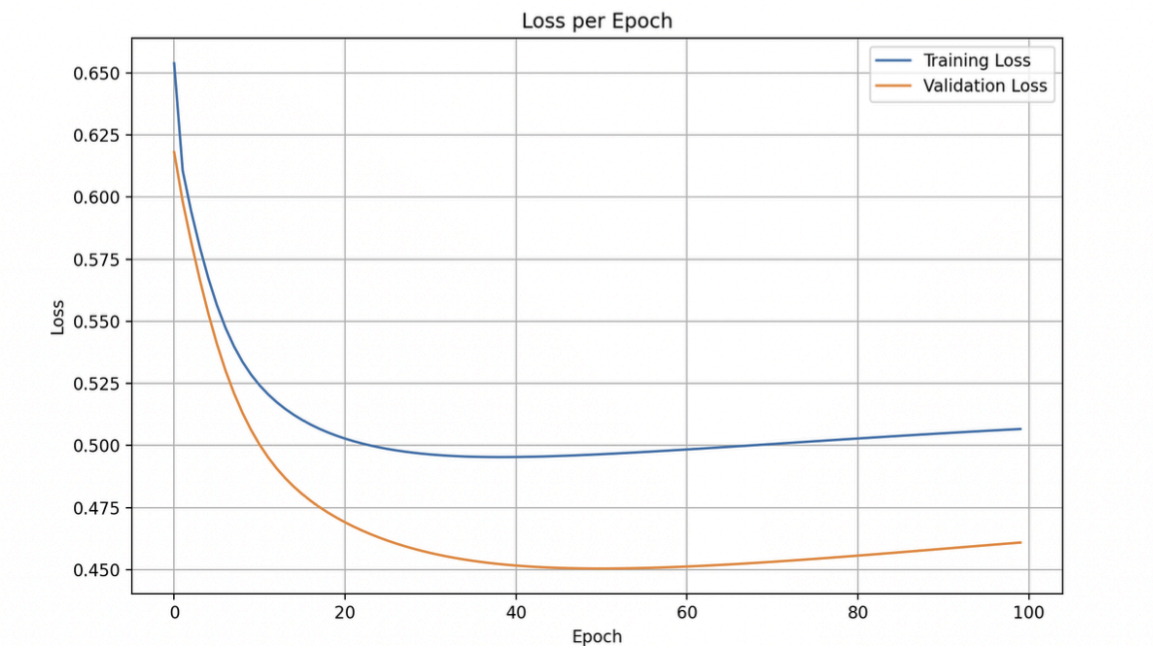
The forward pass is to calculate the activation of hidden and output layers while for backpropagation, gradient descent is implemented to calculate the gradients. z1 is the activation for the hidden layer of one row of data and then will be passed to calculate the sum of activations before applying the sigmoid activation function. The calculated y is the predicted output for x. Backpropagation computes the loss and propagates the error backward to adjust weights and biases. This is to minimise the loss as much as possible by updating the weights and biases.

Model Training and Evaluation

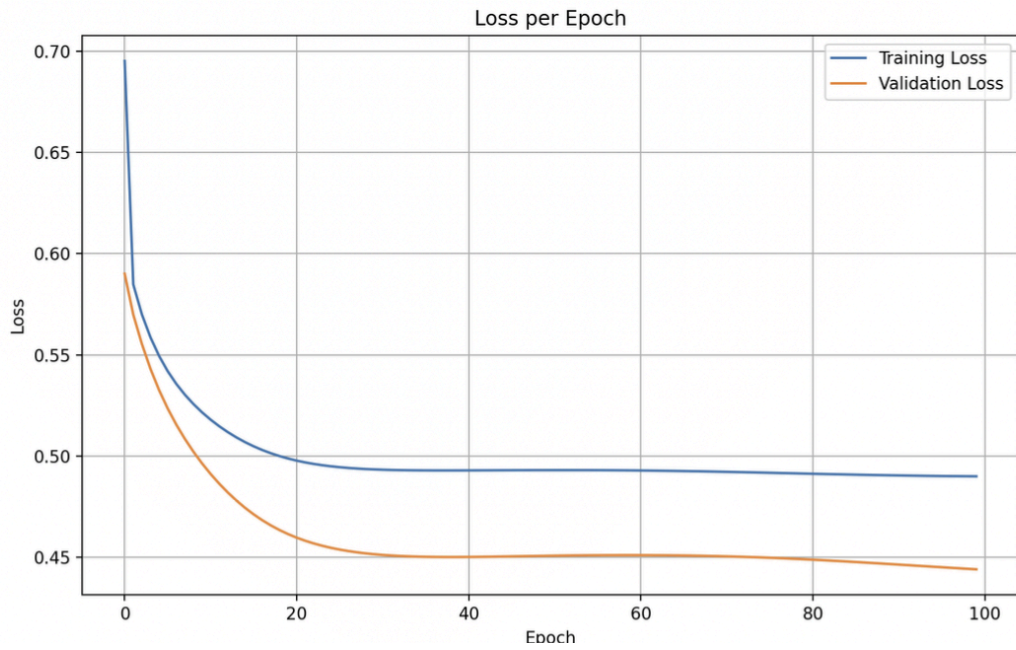
The model will be experimented using different values of learning rate, number of hidden neurons, number of epochs and activation functions. The classification reports for different combinations of hyperparameters values are tabulated below.

	hidden_neurons	Learning rate	epochs	Activation function
Model 1	2	0.01	100	sigmoid
Model 2	4	0.01	100	sigmoid
Model 3	4	0.002	100	sigmoid
Model 4	4	0.002	100	relu
Model 5	4	0.002	1000	relu
Model 6	4	0.002	100	identity
Model 7	4	0.002	100	tanh
Model 8	10	0.002	100	relu
Model 9	4	0.0002	100	identity

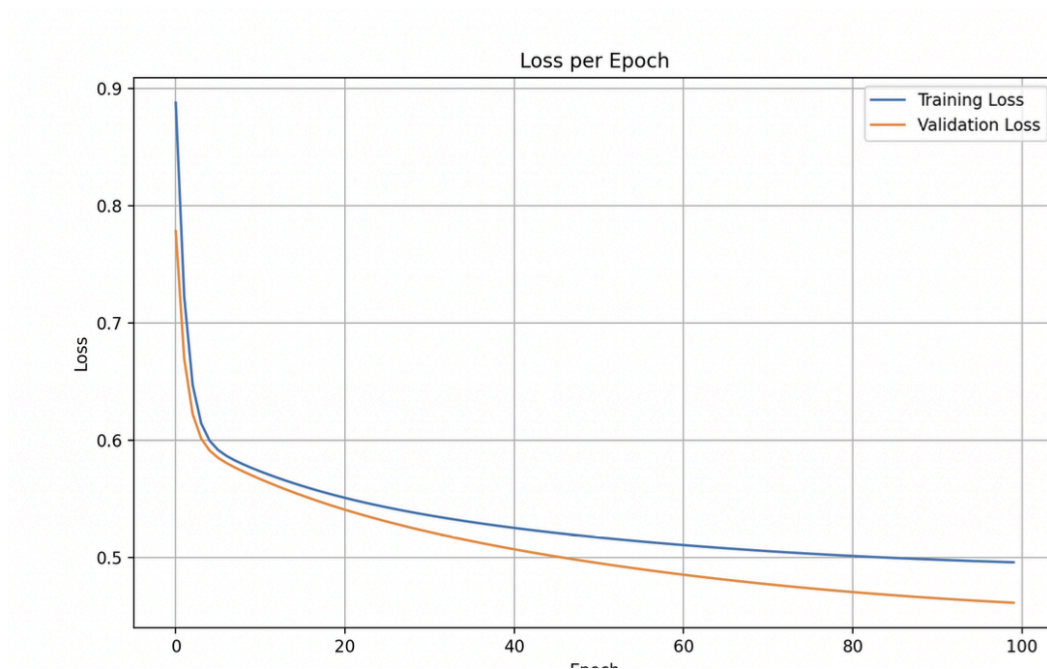
Model 1				
	precision	recall	f1-score	support
0	0.81	0.69	0.74	99
1	0.56	0.71	0.62	55
accuracy	0.69			154
Macro avg	0.68	0.70	0.68	154
Weighted avg	0.72	0.69	0.70	154
Confusion matrix	[[68 31] [16 39]]			



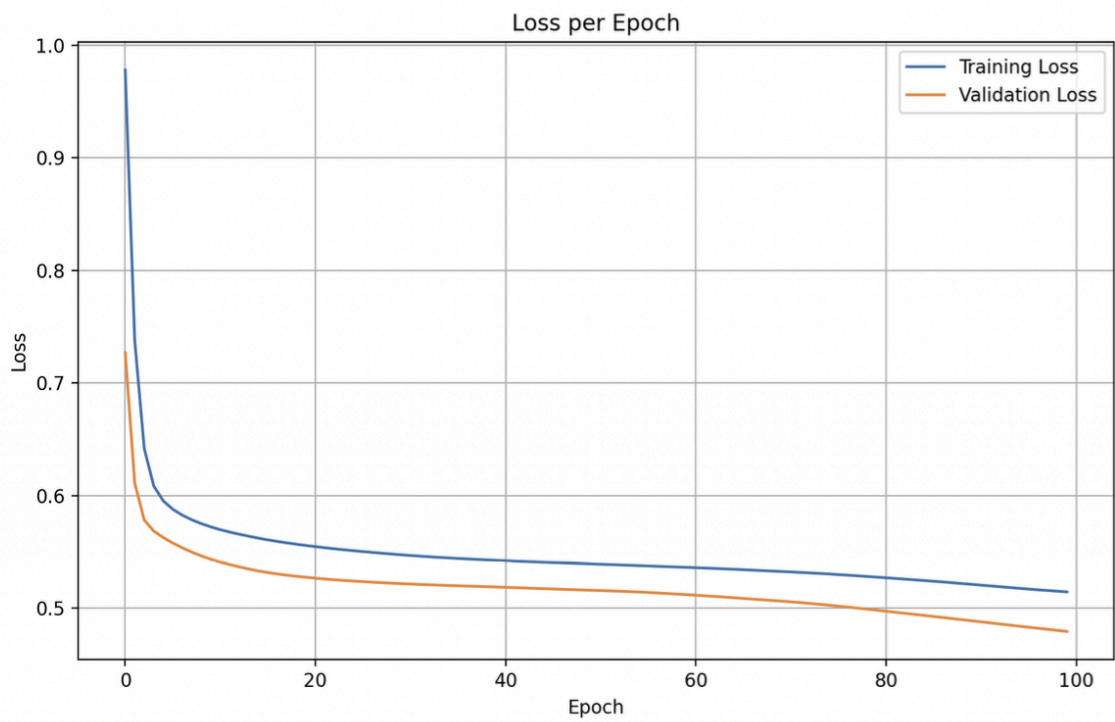
Model 2				
	precision	recall	f1-score	support
0	0.81	0.73	0.77	99
1	0.58	0.69	0.63	55
accuracy	0.71			154
Macro avg	0.70	0.71	0.70	154
Weighted avg	0.73	0.71	0.72	154
Confusion matrix	[[72 27] [17 38]]			



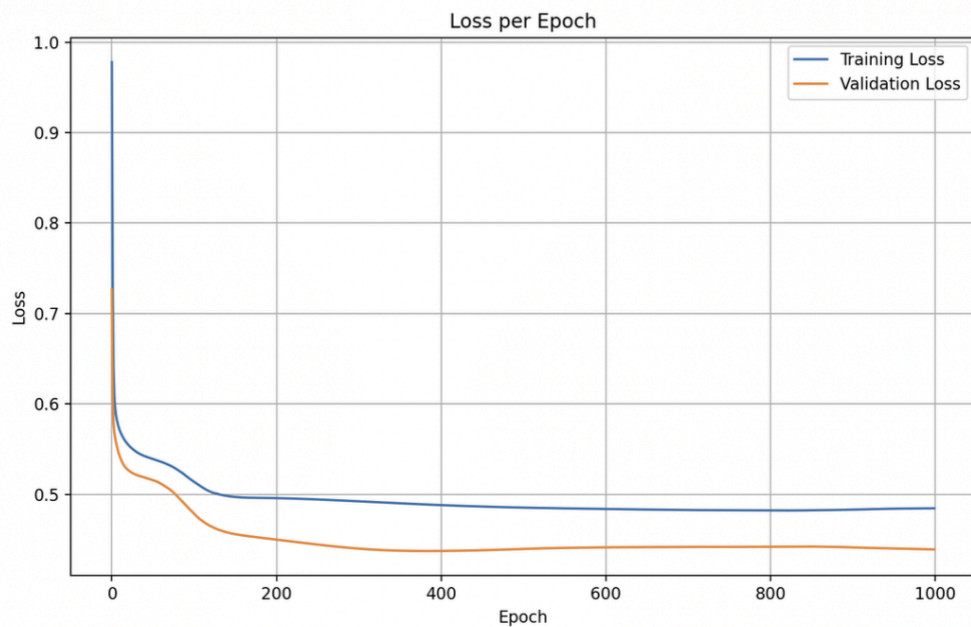
Model 3				
	precision	recall	f1-score	support
0	0.80	0.76	0.78	99
1	0.60	0.65	0.63	55
accuracy	0.72			154
Macro avg	0.69	0.71	0.70	154
Weighted avg	0.73	0.71	0.72	154
Confusion matrix	[[75 24] [19 36]]			



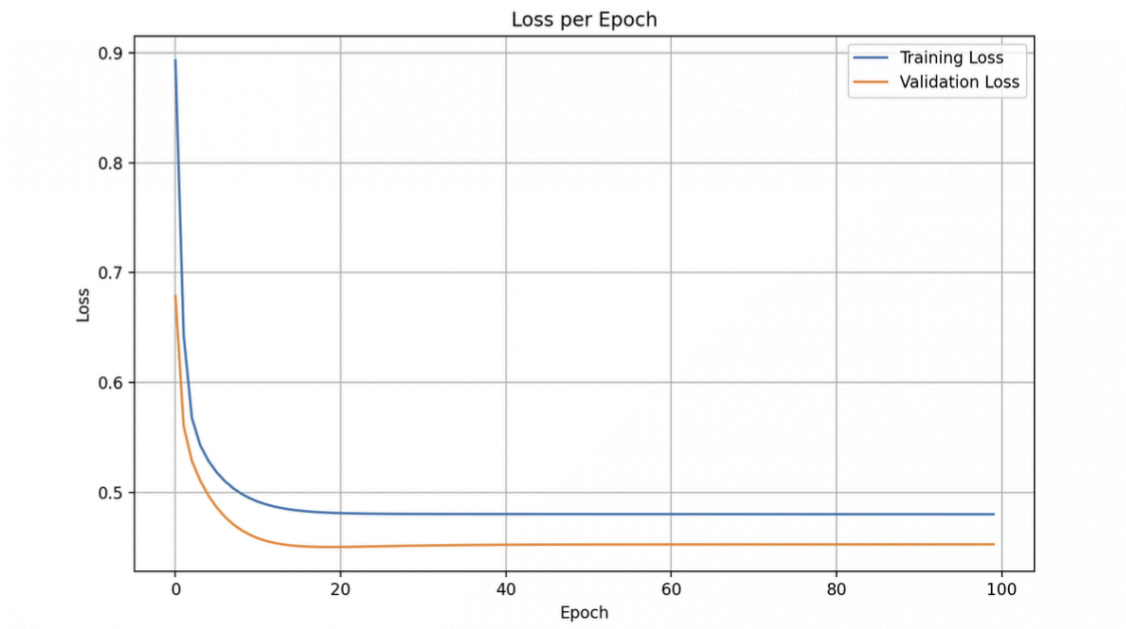
Model 4				
	precision	recall	f1-score	support
0	0.76	0.83	0.79	99
1	0.63	0.65	0.57	55
accuracy	0.72			154
Macro avg	0.69	0.68	0.68	154
Weighted avg	0.71	0.72	0.71	154
Confusion matrix	[[82 17] [26 29]]			



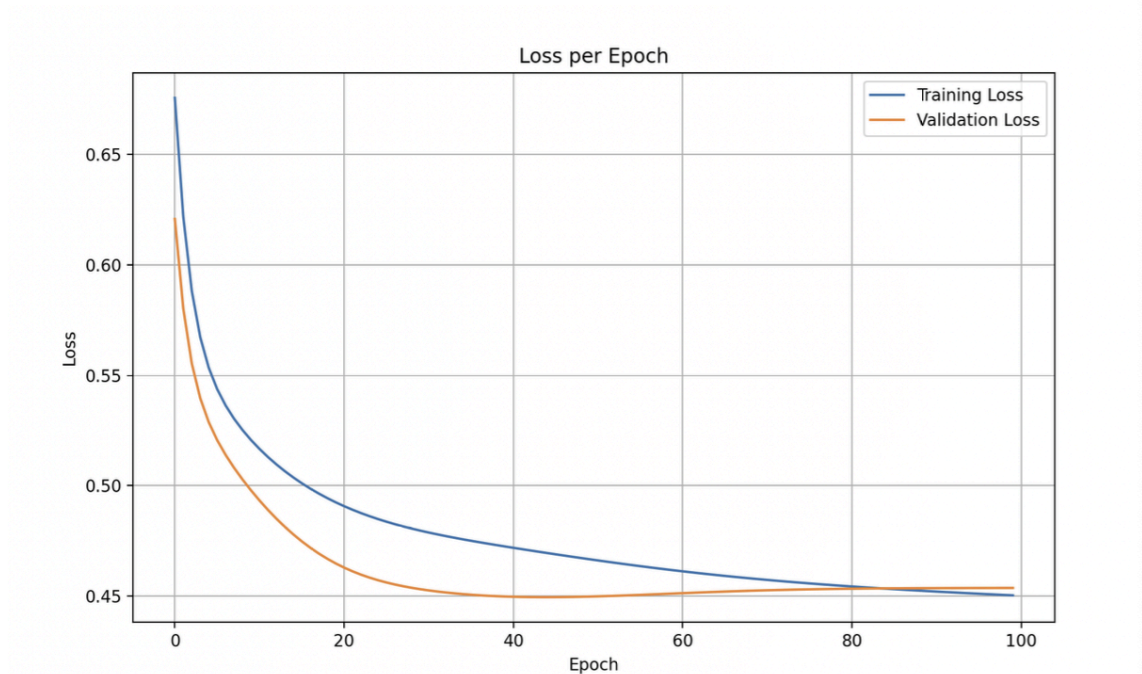
Model 5				
	precision	recall	f1-score	support
0	0.80	0.75	0.77	99
1	0.60	0.67	0.63	55
accuracy	0.72			154
Macro avg	0.70	0.71	0.70	154
Weighted avg	0.73	0.72	0.72	154
Confusion matrix	[[74 25] [18 37]]			



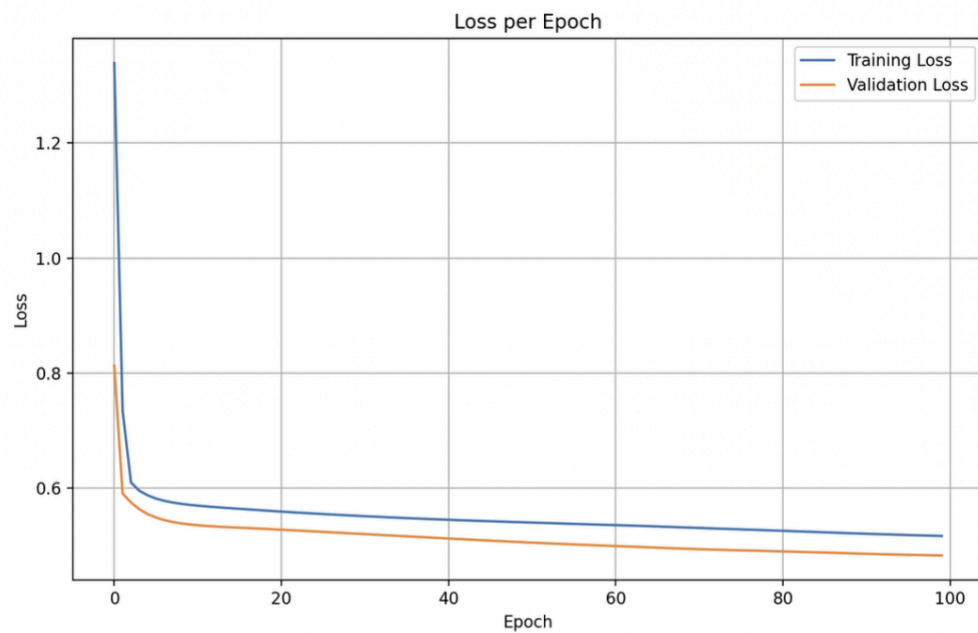
Model 6				
	precision	recall	f1-score	support
0	0.80	0.77	0.78	99
1	0.61	0.65	0.63	55
accuracy	0.73			154
Macro avg	0.71	0.71	0.71	154
Weighted avg	0.73	0.73	0.73	154
Confusion matrix	[[76 23] [19 36]]			



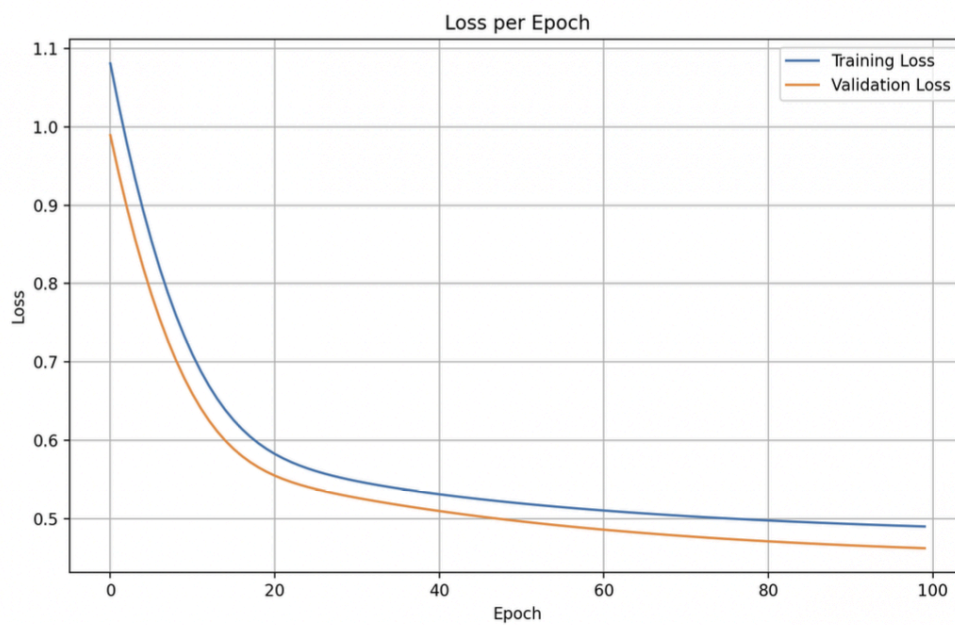
Model 7				
	precision	recall	f1-score	support
0	0.79	0.79	0.79	99
1	0.62	0.62	0.62	55
accuracy	0.73			154
Macro avg	0.70	0.70	0.70	154
Weighted avg	0.73	0.73	0.73	154
Confusion matrix	[[78 21] [21 34]]			



Model 8				
	precision	recall	f1-score	support
0	0.78	0.82	0.80	99
1	0.64	0.58	0.61	55
accuracy	0.73			154
Macro avg	0.71	0.70	0.70	154
Weighted avg	0.73	0.73	0.73	154
Confusion matrix	[[81 18] [23 32]]			



Model 9				
	precision	recall	f1-score	support
0	0.82	0.85	0.84	99
1	0.71	0.67	0.69	55
accuracy	0.79			154
Macro avg	0.77	0.76	0.76	154
Weighted avg	0.78	0.79	0.78	154
Confusion matrix	[[84 15] [18 37]]			



The tuning of hyperparameters starts with 2 hidden neurons, 0.01 of learning rate, 100 epochs with sigmoid activation function. The loss over epochs is not decreasing with accuracy at 0.69, hence, the number of hidden_neurons is increased to 4. The loss over epochs has converged with a slight increase of accuracy to 0.71, but since the gap between training loss and validation loss is high, the tuning of hyperparameters continues. The learning rate is set to 0.002. The loss over epochs starts to decrease with slight increase of accuracy to 0.72, and so to minimise the loss as much as possible, the activation function is changed to 'relu'.

The loss over epochs has reduced after changing the activation function but the performance of the model does not change much in which the accuracy is maintained at 0.72, the epochs are then tuned to 1000 to further minimise the loss. The performance of the model is still maintained after tuning the number of epochs with 0.72 accuracy, the number of epochs is then tuned back to 100 as the gap between two lines increases. Two different activation functions are used, 'identity' and 'tanh', both do not bring much impact besides increase slightly the accuracy to 0.73. Increase of hidden_neurons to 10 with activation function of 'relu' does not alter the performance of model that much, hence, the hidden_neurons is set back to 4 with activation function of 'identity' with learning rate at 0.0002.

The lower the loss curve indicates the better the model in capturing the unseen underlying data pattern, with a minimal gap between training loss and validation loss to avoid overfitting. Based on the loss curve, model 9 has the lowest loss with the least gap between training loss and validation loss. This infers that model 9 has the best performance and generalization among all.

Classification report is used for the view of additional insights of each model, high value of metrics indicates the good performance of the specific model. By comparing all the classification reports, model 9 is still performing the best overall, proven by the accuracy at 0.79 which is the highest among all. Besides, the precision, recall, f1-score, macro avg and weighted avg metrics are the highest among all models too, inferring that this model can correctly classify new samples into true positive and false negative at higher rate and be able to treat both classes equally.

Model Comparison

The classification report for logistic regression, perceptron and MLP are tabulated as below for comparison purposes.

Logistic regression			
	precision	recall	f1-score
0	0.81	0.80	0.81
1	0.65	0.67	0.66
Accuracy			0.75
Macro avg	0.73	0.74	0.73
Weighted avg	0.76	0.75	0.75

Model 6 (Perceptron)			
	precision	recall	f1-score
0	0.82	0.73	0.77
1	0.59	0.71	0.64
Accuracy			0.72
Macro avg	0.70	0.72	0.71
Weighted avg	0.74	0.72	0.73

Model 9 (MLP)			
	precision	recall	f1-score
0	0.82	0.85	0.84
1	0.71	0.67	0.69
Accuracy			0.79
Macro avg	0.77	0.76	0.76
Weighted avg	0.78	0.79	0.78

Conclusion

Based on the analysis of the classification report, MLP is performing better than logistic regression and perceptron model. The precision, f1-score, accuracy, macro avg and weighted avg are the highest, evident that the MLP model is more effective in performing classifying tasks for this dataset.

All in all, multi-layer perceptrons produce better results. Hence, multi-layer perceptron is the better choice for this classification task due to the fact that the hyperparameters of this model can be tuned in order to train a model that fits to the dataset and produce a reliable result.